



大模型 Prompt 工程全指南 (进阶详尽版)

第一部分：Prompt 核心定义与原理

1. 什么是 Prompt?

Prompt (提示词) 是在大语言模型 (LLM) 应用中，用于“提示”模型唤起特定能力以解决实际问题的输入方式。

- **本质：** 它并非简单的提问，而是引导模型从预训练阶段积累的海量信息中，精准唤起理解、总结、生成、逻辑推理等能力的“钥匙”。
- **核心作用：** 高效调用模型智能，实现类似人类解决复杂问题的效果，而非将其视为单纯的静态知识库。

Prompt 工程核心原则 (3分钟速通版)

想要训练模型，你需要遵循 **C-R-I-S-P** 五原则：

1. **C - Context (背景)：** 模型不知道你是谁。给它一个身份（资深律师、雅思考官）和具体的任务背景。
2. **R - Reference (参考)：** 哪怕是几行示例 (Few-Shot) 也比没有强。给模型看“好的标准”是什么。
3. **I - Instruction (指令)：** 动词要狠。用“总结”、“改写”、“提取”而不是“帮我看看”。
4. **S - Structure (结构)：** 使用分隔符（如 ### 或 ---）把背景、指令、示例分开，防止模型“串味”。
5. **P - Precision (精确)：** 拒绝“长一点”或“好一点”。说“500字以内”、“三段式布局”或“幽默风格”。

核心思维转变： 不要把 AI 当作搜索引擎，要把 AI 当作一个**极其聪明但没有常识、需要明确指令的实习生**。

三大关键法则

1. 消除模糊性 (Be Specific)：

- **✗ 差：** 帮我写个文案。
- **✓ 优：** 你是一位小红书爆款写手 (Role)。请为一款美白面膜写一篇种草文案 (Task)。受众是经常熬夜的大学生 (Context)。要求语气活泼，多用Emoji，并在文末加上购买链接引导 (Constraint)。

2. 给模型“思考时间” (Chain of Thought)：

- 大模型像一个反应极快的直觉型选手，容易算错或逻辑跳跃。
- **✓ 技巧：** 在指令里强制加入“**请一步步思考 (Let's think step by step)**”，或者明确拆解步骤（第一步...第二步...），能让逻辑题准确率飙升。

3. 喂饭式教学 (Few-Shot Prompting)：

- 这是很多人的盲区。与其用 1000 字描述你要的格式，不如直接给它 1-2 个“**输入-输出**”的完美范例。
- **✓ 技巧：** 给它看一个你写好的标准答案，告诉它“就像这样写”。

第二部分：十大主流撰写框架（含深度解析与示例）

1. The Role-Play Expert（角色沉浸模版）

- 💡 **使用场景：** 需要特定领域的专业知识、语气或视角时（如法律咨询、医疗科普、客户服务）。
- 📄 **模板结构：**

Markdown

```
# Role
你是一位[具体角色，如：拥有15年经验的雅思口语考官]。你的特点是[性格/语气，如：严厉但富有建设性，喜欢使用地道的英式习语]。

# Task
请针对以下[输入内容]，以你的专业视角进行[具体动作，如：点评并给出修改建议]。

# Constraints
- 必须指出3个具体的错误。
- 给出一段满分范例。
- 保持语气专业。

# Input Data
[用户输入的内容]
```

- 📄 **示例输入：** "Task: 点评我的英语自我介绍: 'Hi, I am Li Hua. I like play basketball...'"
- ✅ **示例输出：** 模拟考官语气，指出语法错误（like play -> like playing），使用英式习语（"keen on"），并给出一个高分版本。

2. The Chain of Thought (CoT) Reasoner（思维链模版）

- 💡 **使用场景：** 复杂的逻辑推理、数学计算、Debug、因果分析。**这是提升模型智商最有效的方法。**
- 📄 **模板结构：**

Markdown

```
# Task
请回答以下问题：[复杂问题描述]

# Instruction
不要直接给出答案。请按照以下步骤逐步思考：
1. 第一步：[分析问题的核心变量]
2. 第二步：[建立逻辑关联/计算过程]
3. 第三步：[排除干扰项]
4. 第四步：[得出最终结论]

请展示你的每一步思考过程。
```

- 📄 **示例输入：** "如果昨天是明天的前三天，那么今天是周几？"
- ✅ **示例输出：** 模型不会乱猜，而是列出方程：设今天是X，明天是X+1，明天的前三天是(X+1)-3... 最终推导出准确答案。

3. The Few-Shot Imitator (少样本模仿模版)

- 💡 **使用场景：** 想要特定的格式、风格、JSON结构，或者模型听不懂指令时（直接喂饭）。
- 📄 **模板结构：**

Markdown

```
# Task
请模仿以下示例的风格/格式，处理新的输入。

# Examples
输入：苹果
输出：🔴 红色 | 水果 | 甜脆
---
输入：天空
输出：🔵 蓝色 | 自然 | 广阔
---

# New Input
输入：[用户的新词]
输出：
```

- 📄 **示例输入：** "输入：法拉利"
- ✅ **示例输出：** "🔴 红色 | 汽车 | 速度"（模型完美捕捉到了：Emoji颜色+类别+形容词的结构）。

4. The Critic & Refiner (批评家与优化师模版)

- 💡 **使用场景：** 写高质量文案、代码审查、优化粗糙的初稿。
- 📄 **模板结构：**

Markdown

```
# Context
我写了一篇[文章/代码]，但我感觉不够好。

# Task 1: Critique
请作为一位[严苛的编辑/资深架构师]，列出这段内容的3个主要缺点（逻辑漏洞、用词重复、效率低下等）。

# Task 2: Refine
根据上述批评意见，重写这段内容，使其达到[具体标准，如：出版级/生产环境可用]。

# Content
[待优化的内容]
```

- 📄 **示例输入：** 一段啰嗦的周报。
 - ✅ **示例输出：** 先列出“缺乏数据支撑、重点不突出”等问题，然后给出一版精炼、带数据的完美周报。
-

5. The Socratic Tutor（苏格拉底教学模版）

- 💡 **使用场景：** 学习新知识。你不想直接要答案，而是想通过对话深入理解。
- 📄 **模板结构：**

Markdown

```
# Role
你是一位苏格拉底式的导师。

# Instruction
我想学习[某个概念]。请不要直接把定义告诉我。
你需要通过通过一系列引导性的问题，启发我自己思考并得出答案。
一次只问一个问题，根据我的回答调整下一个问题。

# Topic
[量子纠缠 / 动态规划 / 市场营销4P理论]
```

- 📄 **示例输入：** "Topic: 什么是'第一性原理'？"
- ✅ **示例输出：** "很好。在讨论定义之前，我想问你：当你想要改进一辆马车时，你会盯着马看，还是盯着运输的本质看？"

6. The Structured Data Extractor（结构化提取模版）

- 💡 **使用场景：** 从长文中提取关键信息，转为 Excel、JSON 或表格。
- 📄 **模板结构：**

Markdown

```
# Task
从以下文本中提取关键实体。

# Output Format
请严格输出为 JSON 格式，包含以下字段：
- "customer_name": 客户姓名
- "pain_point": 痛点（不超过10个字）
- "budget": 预算（若未提及填 null）

# Text
[一段混乱的会议记录]
```

- 📄 **示例输入：** 客服对话记录。
 - ✅ **示例输出：** `json {"customer_name": "张三", "pain_point": "电池续航短", "budget": "5000元"}`
-

7. The Style Transformer (风格转换模版)

- 💡 **使用场景：** 将一段话改写成完全不同的受众版本（如：把技术文档改成给奶奶看的信）。
- 📄 **模板结构：**

Markdown

```
# Original Text
[一段复杂的文本]

# Goal
将上述文本改写。

# Target Audience
[目标受众: 5岁儿童 / 专业的博士 / 愤怒的客户]

# Tone Guidelines
- 使用[简单的比喻 / 严谨的数据 / 极其抱歉的态度]。
- 避免[专业术语 / 废话]。
```

- 📄 **示例输入：** 一段关于区块链的定义。
- ✅ **示例输出：** (针对5岁儿童) "想象你有一个特殊的笔记本，大家都可以往上写字，但是一旦写上去，就再也擦不掉了..."

8. The Code Master (代码生成与解释模版)

- 💡 **使用场景：** 写代码、解释代码、修复 Bug。
- 📄 **模板结构：**

Markdown

```
# Language
[Python / JavaScript / SQL]

# Task
编写一个函数来实现[具体功能]。

# Requirements
1. 代码必须包含详细的注释。
2. 考虑[边缘情况，如：输入为空]。
3. 时间复杂度最优。

# Explanation
代码结束后，请用通俗语言解释核心逻辑。
```

- 📄 **示例输入：** "写一个快速排序算法。"
 - ✅ **示例输出：** 标准的代码实现，加上了防错处理，并在最后解释了“分治法”的原理。
-

9. The Brainstorming Expander (发散创意模版)

- 💡 **使用场景：** 没灵感、起标题、想活动方案。
- 📄 **模板结构：**

Markdown

```
# Context
我要策划一个[主题]活动。

# Task
请提供 10 个创意方向。

# Constraints (重要!)
- 3个方案要是疯狂的、不计成本的。
- 3个方案要是零成本的。
- 4个方案要结合当前的流行梗。

请用列表形式展示，每个方案附带一句Slogan。
```

- 📄 **示例输入：** "公司年会策划"
- ✅ **示例输出：** 既有“包下迪士尼”的疯狂方案，也有“办公室吐槽大会”的零成本方案，极大地打开了思路。

10. The Meta-Prompt (让 AI 写 Prompt 的模版)

- 💡 **使用场景：** 终极绝招。当你不知道怎么写 Prompt 时，让 AI 帮你写。
- 📄 **模板结构：**

Markdown

```
# Role
你是一位 Prompt Engineering 专家。

# Task
我想让 AI [描述你的模糊需求，如：帮我自动生成PPT大纲]。
请帮我撰写一个高质量的 Prompt，让我可以直接发给 ChatGPT 使用。

# Requirements
- 包含 Role, Task, Constraints, Output Format。
- 使用结构化的 Markdown 格式。
```

- 📄 **示例输入：** "我想让 AI 帮我把大白话翻译成古文。"
- ✅ **示例输出：** AI 会吐出一个完美的 Prompt，你直接复制那个 Prompt 去用即可。

第三部分：打造高效 Prompt 的自检清单

✅ 基础完备性检查 (The Core 4)

如果不满足这 4 点，AI 大概率会给你一个平庸的通用回答。

- ☐ **角色 (Role) 是否具体？**
 - ❌ 差：你是一个助手。
 - ✅ 优：你是一位拥有 10 年经验的 Python 后端架构师 / 擅长情感营销的小红书博主。
 - ☐ **背景 (Context) 是否给够？**
 - ❌ 差：帮我写封邮件。
 - ✅ 优：我是一名求职者，需要给面试官写一封感谢信，面试职位是产品经理，面试过程中我们聊到了 AI 趋势...
 - ☐ **指令 (Task) 是否使用了强动词？**
 - ❌ 差：帮我看看这个... / 我想问一下...
 - ✅ 优：请**分析**... / 请**重写**... / 请**提取**... / 请**对比**...
 - ☐ **输出格式 (Output) 是否明确？**
 - ❌ 差：把结果发我。
 - ✅ 优：请用 **Markdown 表格**展示 / 请以 **JSON 格式**输出 / 请分点列出，每点不超过 50 字。
-

🚀 效果增强检查 (Quality Boosters)

加上这几项，AI 会变得更聪明、更懂你。

- ☐ **是否有约束条件 (Constraints)？**
 - 是否限制了字数（如：200字以内）？
 - 是否规定了“**不要做什么**”（如：不要使用敬语、不要包含代码解释）？
 - ☐ **是否提供了示例 (Few-Shot)？**
 - **(最重要的一项！)** 是否给了 1-2 个“输入+理想输出”的例子？哪怕只有一个例子，也能大幅修正 AI 的风格。
 - ☐ **是否开启了“思维链” (CoT)？**
 - 如果是逻辑/计算/分析任务，是否加上了“**请逐步思考**” (Let's think step by step) 或拆解了步骤？
 - ☐ **语气风格 (Tone) 是否指定？**
 - 是严肃专业、幽默风趣、还是通俗易懂？
-

🛡️ 逻辑与安全检查 (Sanity Check)

防止 AI 胡说八道或格式混乱。

- ☐ **结构是否清晰？**
 - 是否使用了分隔符（如 `###`、`---`、`"""`）将指令、背景和数据隔开？
- ☐ **是否进行了反向确认？**

- 对于极重要的任务，是否加了一句：“在执行前，如果你对我的需求有不清楚的地方，请向我提问。”

⚡ 一句话自测法 (The One-Sentence Test)

问自己：

“如果我把这个 Prompt 发给一个完全不懂我业务的新实习生，他能不问我任何问题，就直接拿出我想要的结果吗？”

- 如果答案是 **NO** → 请补充背景或约束。
- 如果答案是 **YES** → 发送！